
AlterDune

Rapport de Projet POO C++

Jean Baptiste PASTEAU & Victor MICHELON

Le projet Alterdune est un mini-jeu de rôle (RPG) jouable dans la console. L'objectif principal de ce projet était de mettre en pratique les concepts fondamentaux de la Programmation Orientée Objet en C++, comme l'encapsulation, l'héritage et le polymorphisme. Le joueur doit affronter des monstres et choisir s'il souhaite les combattre ou les épargner, ce qui mène à différentes fins de partie.

1. Fonctionnement du jeu et règles

1.1 – Objectifs et Conditions de victoire

Une partie se joue en 10 combats. À chaque combat gagné, le compteur de victoires augmente d'un point. Quand il atteint 10, la partie se termine et le jeu affiche une fin différente selon le comportement du joueur :

- Fin Pacifiste : le joueur a épargné toutes les créatures rencontrées.
- Fin Génocidaire : tous les monstres ont été battu par le joueur.
- Fin Neutre : le joueur a tué et épargné des monstres.
- Mort du Joueur : le joueur est battu par un monstre, il n'a plus de points de vie

1.2 – Déroulement de la partie

Au lancement, le programme demande un nom de personnage puis charge automatiquement la base de données `items.csv` pour l'inventaire de départ et `monsters.csv` pour les monstres disponibles. Une fois les données chargées, le menu principal s'affiche.

Depuis le menu principal, le joueur peut :

- Lancer un combat contre un monstre tiré aléatoirement
- Consulter le bestiaire (la liste de tous les monstres déjà rencontrés)
- Voir ses statistiques personnelles et sa jauge de progression
- Accéder à son inventaire et utiliser des objets de soin

- Quitter la partie

1.3 – Système de Combat

Chaque combat est au tour par tour. Le joueur agit en premier, puis le monstre riposte s'il est encore vivant. À son tour, le joueur choisit parmi quatre actions : Fight, Act, Item, Mercy. Respectivement une attaque directe, une interaction disponible pour ce monstre qui fait évoluer la jauge de Mercy, utilise un objet de l'inventaire pour regagner des PV et consomme le tour, épargne le monstre si la jauge Mercy est pleine

2. Architecture et Modélisation POO

Pour structurer le code, nous avons découpé le projet en plusieurs classes logiques.

2.1 – Héritage des entités

J'ai créé une classe abstraite de base appelée Entity. Elle contient les attributs communs à tous les personnages : le nom, les HP, l'attaque et la défense. Elle gère aussi la prise de dégâts et le soin. La classe Player hérite de Entity. Elle ajoute la gestion de l'inventaire (une liste d'objets Item) et compte le nombre de victoires.

La classe Monster hérite aussi de Entity. Elle ajoute le système de Mercy et la liste des actions ACT que le monstre peut subir.

2.2 – Les polymorphisme des monstres

Les monstres sont divisés en trois catégories dérivées de la classe Monster : NormalMonster, MinibossMonster et BossMonster. Chacune de ces classes redéfinit la méthode getActCapacity() pour déterminer combien d'actions sont proposées au joueur (2, 3 ou 4).

2.3 – La gestion des systèmes

La classe Game est le moteur du jeu. Elle gère la boucle principale, déclenche les combats et relie le joueur aux monstres. La classe FileLoader s'occupe de lire les fichiers items.csv et monsters.csv pour charger les données de départ. La classe DisplayManager est responsable de tout l'affichage visuel du jeu.

3. Choix d'Implémentation et Justifications

3.1 – La logique d'attaque (Dégâts)

Le sujet demandait initialement de calculer les dégâts avec une formule aléatoire basée sur les HP maximum du défenseur (). L'attaque du monstre est tiré aléatoirement entre 0 son score d'attaque ATK. Ce choix permet de rendre le jeu beaucoup plus cohérent pour un RPG. Ainsi, les statistiques d'attaque inscrites dans le fichier `monsters.csv` servent réellement à quelque chose et créent une vraie courbe de difficulté.

3.2 – Le Design Pattern « Prototype » (Clonage)

Plutôt que d'utiliser directement les monstres chargés depuis le fichier, nous avons implémenté une méthode `clone()` dans les classes de monstres.

Cela permet de créer une copie neuve d'un monstre lorsqu'un combat démarre. Ainsi, la base de données d'origine reste intacte dans la base de données, ce qui permet de réaffronter le même type de monstre plusieurs fois sans qu'il ne commence le combat déjà blessé.

4. Difficultés Rencontrées et Solutions

Difficulté 1 : L'épuisement des monstres

Au début, quand le joueur gagnait un combat, le monstre était retiré de la liste. Si le fichier CSV contenait peu de monstres, le jeu bloquait avant d'atteindre les 10 victoires.

Solution : Comme expliqué plus haut, j'ai utilisé la méthode `clone()` pour copier les monstres au lieu de les retirer de la liste principale (`monsterPool`).

Difficulté 2 : Le décalage des cadres d'affichage

Pour rendre le jeu visuellement agréable, j'ai créé des cadres avec le caractère `|`. Cependant, l'utilisation de couleurs (codes ANSI) et de caractères accentués décalait les cadres, car le C++ comptait mal la longueur visuelle des mots.

Solution : J'ai développé une fonction personnalisée `visLen()` dans `DisplayManager` qui ignore les codes d'échappement de couleur et les octets spéciaux des accents (UTF-8) lors du calcul des espaces. J'ai aussi simplifié les textes finaux pour éviter les caractères trop exotiques.

Difficulté 3 : La gestion de l'aléatoire

La fonction basique `rand()` générait toujours la même suite de combats et de dégâts à chaque lancement du jeu.

Donc nous avons remplacé `rand()` par la bibliothèque `<random>` du C++ moderne. Nous utilisons « `std::mt19937` » pour générer un aléatoire robuste et « `std::uniform_int_distribution` » pour que les dégâts et le choix des actions du monstre soient vraiment imprévisibles.